

解题报告

姓名：白家辉

班级：计算机大类2508

学号：8208250831

日期：2026.6.18

总览

本次解题报告中，共完成 2 道题目。

题目名称	难度	知识点
P2984 Chocolate Giving S	普及/提高-	最短路、Dijkstra、路径拆分
P8802 出差	普及/提高-	最短路、点权转边、手写堆

（一）Chocolate Giving S

题目信息

题目链接：<https://www.luogu.com.cn/problem/P2984>

知识点：最短路、Dijkstra、路径拆分

题目简述

农场里有 N 个牧场、 M 条双向道路，以及 B 组“公牛送巧克力”请求。每头公牛需要先从所在牧场走到 1 号牧场拿巧克力，再从 1 号牧场走到目标母牛所在牧场。

对于全部数据， $2 * B \leq N \leq 50000$ ， $N - 1 \leq M \leq 10^5$ ， $B \leq 25000$ ，道路长度不超过 2000。

题目分析

1. 读题

每个询问看起来都像一条新的最短路：公牛从 p_i 出发，到 1 号点，再到 q_i 。如果对每组询问都单独跑一遍最短路，显然会重复计算很多次。

2. 性质观察

题目强制所有路线都必须经过 1 号牧场，因此一条合法路线的长度一定是：

$$\text{dist}(P_i, 1) + \text{dist}(1, Q_i)$$

又因为图是无向图，所以 $\text{dist}(P_i, 1) = \text{dist}(1, P_i)$ 。于是只要从 1 号点做一次单源最短路，之后每个询问都可以直接用：

$$\text{dist}[P_i] + \text{dist}[Q_i]$$

得到答案。

3. 程序设计思路

1. 用链式前向星存储无向图；
2. 从 1 号点运行一次 Dijkstra；
3. 当前代码使用的是手写二叉堆，而不是库优先队列；
4. 求出 1 到所有点的最短距离后，依次处理每组询问；
5. 对每组 (P_i, Q_i) 直接输出 $\text{dist}[P_i] + \text{dist}[Q_i]$ ；
6. 因为路径长度可能较大，所以 dist 使用 `long long`。

4. 数据结构与算法选择

本题最关键的不是多次最短路，而是发现“所有询问都共享同一个中转点 1”。当前实现因此采用“一次手写堆 Dijkstra + 多次 $O(1)$ 回答询问”的方式。

5. 时空复杂度分析

- 时间复杂度： $O((N + M) \log N + B)$ 。
- 空间复杂度： $O(N + M)$ 。

首页 / 评测记录 / 评测详情

R282396003 记录详情

编程语言

C O2

代码长度

2.06KB

用时

88ms

内存

4.73MB

测试点信息

源代码

测试点信息

#1

AC

4ms/2.28MB

#2

AC

4ms/2.25MB

#3

AC

4ms/2.52MB

#4

AC

4ms/2.52MB

#5

AC

4ms/2.50MB

#6

AC

6ms/4.31MB

#7

AC

6ms/4.28MB

#8

AC

10ms/4.27MB

#9

AC

28ms/4.73MB

#10

AC

18ms/4.25MB

jsj0708Baijiahui

所属题目

P2984 [USACO10FEB] Chocolat...

评测状态

Accepted

评测分数

100

提交时间

2026-06-20 12:34:11

```
#include <stdio.h>
#include <string.h>

#define N 50005
#define M 200005
#define INF 0x3f3f3f3f3f3f3fLL

typedef long long ll;

typedef struct
{
    int to;
    int w;
    int next;
}Edge;

Edge edge[M];

int head[N];
int idx;

void add(int u,int v,int w)
{
    edge[idx].to=v;
    edge[idx].w=w;
    edge[idx].next=head[u];
    head[u]=idx++;
}

ll dist[N];
int vis[N];

typedef struct
{
    int id;
    ll dis;
}Node;

Node heap[M];
int size;

void swap(Node *a,Node *b)
{
```

```

    Node t=*a;
    *a=*b;
    *b=t;
}

void push(Node x)
{
    heap[++size]=x;

    int i=size;

    while(i>1)
    {
        int fa=i/2;

        if(heap[fa].dis<=heap[i].dis)
            break;

        swap(&heap[fa],&heap[i]);
        i=fa;
    }
}

Node pop()
{
    Node ret=heap[1];

    heap[1]=heap[size--];

    int i=1;

    while(1)
    {
        int l=i*2;
        int r=i*2+1;
        int smallest=i;

        if(l<=size && heap[l].dis<heap[smallest].dis)
            smallest=l;

        if(r<=size && heap[r].dis<heap[smallest].dis)
            smallest=r;
    }
}

```

```

        if(smallest==i)
            break;

        swap(&heap[i],&heap[smallest]);
        i=smallest;
    }

    return ret;
}

void dijkstra(int n)
{
    memset(vis,0,sizeof(vis));

    for(int i=1;i<=n;i++)
        dist[i]=INF;

    dist[1]=0;

    push((Node){1,0});

    while(size)
    {
        Node cur=pop();

        int u=cur.id;

        if(vis[u])
            continue;

        vis[u]=1;

        for(int i=head[u];i!=-1;i=edge[i].next)
        {
            int v=edge[i].to;
            int w=edge[i].w;

            if(dist[v]>dist[u]+w)
            {
                dist[v]=dist[u]+w;
                push((Node){v,dist[v]});
            }
        }
    }
}

```

```

    }
}

int main()
{
    int n,m,b;

    scanf("%d%d%d",&n,&m,&b);

    memset(head,-1,sizeof(head));

    for(int i=0;i<m;i++)
    {
        int u,v,w;

        scanf("%d%d%d",&u,&v,&w);

        add(u,v,w);
        add(v,u,w);
    }

    dijkstra(n);

    while(b--)
    {
        int p,q;

        scanf("%d%d",&p,&q);

        printf("%lld\n",dist[p]+dist[q]);
    }

    return 0;
}

```

(二) 出差

题目信息

题目链接：<https://www.luogu.com.cn/problem/P8802>

知识点：最短路、点权转边、手写堆

题目简述

需要从城市 1 赶到城市 N 。道路有通行时间，此外到达某个中转城市后还需要隔离一段时间，只有城市 1 不需要出发前隔离，城市 N 到达后也不需要再计算隔离时间。

对于全部数据， $N \leq 1000$ ， $M \leq 10000$ ，隔离时间 $C_i \leq 200$ ，道路耗时 $c \leq 1000$ 。

1.00s, 128MB。

题目分析

1. 读题

题目表面是最短路，但路径代价不只来自边权，还来自中转城市的隔离时间。也就是说，这是一道“边权 + 点权”共同参与的最短路问题。

2. 性质观察

对于一条路径，总花费等于：

1. 所有经过道路的通行时间之和；
2. 所有中间停留城市的隔离时间之和；
3. 但起点 1 不额外加隔离，终点 N 到达后也不需要再隔离。

这说明可以把“进入城市 v 的代价”改写为：

道路时间 + $C[v]$

其中当 $v = N$ 时，不再加 $C[v]$ 。经过这样转换后，问题就重新变成了标准的非负权最短路。

3. 程序设计思路

- 1. 读入每个城市的隔离时间数组 iso 和所有双向道路；
- 2. 用邻接表建图；
- 3. 初始化 dist[1]=0，其余为无穷大；
- 4. 使用手写小根堆执行 Dijkstra；
- 5. 从 u 松弛到 v 时，先取道路时间 w；
- 6. 如果 v 不是终点 n，就再额外加上 iso[v]；
- 7. 若新代价更优，就更新并入堆；
- 8. 最终输出 dist[n]。

4. 数据结构与算法选择

本题依旧适合 Dijkstra，因为所有通行时间和隔离时间都非负。当前代码同样采用手写堆实现，关键点是把中转城市隔离时间正确并入边松弛，而终点城市不再额外加隔离代价。

5. 时空复杂度分析

- 时间复杂度： $O((N + M) \log N)$ 。
- 空间复杂度： $O(N + M)$ 。

代码实现

洛谷 / 评测记录 / 评测详情

R282396049 记录详情

编程语言C++2代码长度2.10KB用时49ms内存824.00KB

测试点信息源代码

测试点信息

#1 AC 5ms/816.00KB	#2 AC 5ms/812.00KB	#3 AC 5ms/820.00KB	#4 AC 4ms/612.00KB	#5 AC 5ms/736.00KB	#6 AC 5ms/816.00KB	#7 AC 5ms/812.00KB
#8 AC 5ms/812.00KB	#9 AC 5ms/704.00KB	#10 AC 5ms/824.00KB				

jsj0708Baijiahui

所属题目P8802 [蓝桥杯 2022 国 B] 出差

评测状态Accepted

评测分数100

提交时间2026-06-20 12:34:46

```
#include <stdio.h>
#include <string.h>

#define N 1005
#define M 20005
#define INF 0x3f3f3f3f

typedef struct {
    int to;
    int w;
    int next;
} Edge;

Edge edge[M];
int head[N], idx;

int iso[N];
int dist[N];
int vis[N];

typedef struct {
    int id;
    int dis;
} Node;

Node heap[M];
int sz;

void add(int u, int v, int w)
{
    edge[idx].to = v;
    edge[idx].w = w;
    edge[idx].next = head[u];
    head[u] = idx++;
}

void swap(Node *a, Node *b)
{
    Node t = *a;
    *a = *b;
    *b = t;
}
```

```

void push(Node x)
{
    heap[++sz] = x;

    int i = sz;
    while (i > 1)
    {
        int fa = i / 2;

        if (heap[fa].dis <= heap[i].dis)
            break;

        swap(&heap[fa], &heap[i]);
        i = fa;
    }
}

```

```

Node pop()
{
    Node ret = heap[1];

    heap[1] = heap[sz--];

    int i = 1;

    while (1)
    {
        int smallest = i;
        int l = i * 2;
        int r = i * 2 + 1;

        if (l <= sz && heap[l].dis < heap[smallest].dis)
            smallest = l;

        if (r <= sz && heap[r].dis < heap[smallest].dis)
            smallest = r;

        if (smallest == i)
            break;

        swap(&heap[i], &heap[smallest]);
        i = smallest;
    }
}

```

```

    return ret;
}

int main()
{
    int n, m;
    scanf("%d%d", &n, &m);

    for (int i = 1; i <= n; i++)
        scanf("%d", &iso[i]);

    memset(head, -1, sizeof(head));

    for (int i = 0; i < m; i++)
    {
        int u, v, w;
        scanf("%d%d%d", &u, &v, &w);

        add(u, v, w);
        add(v, u, w);
    }

    memset(dist, 0x3f, sizeof(dist));

    dist[1] = 0;

    push((Node){1, 0});

    while (sz)
    {
        Node cur = pop();
        int u = cur.id;

        if (vis[u])
            continue;

        vis[u] = 1;

        for (int i = head[u]; i != -1; i = edge[i].next)
        {
            int v = edge[i].to;

```

```
    int cost = edge[i].w;

    if (v != n)
        cost += iso[v];

    if (dist[v] > dist[u] + cost)
    {
        dist[v] = dist[u] + cost;
        push((Node){v, dist[v]});
    }
}

printf("%d\n", dist[n]);

return 0;
}
```